

Epílogo – Proyecto Peluquería

Este proyecto consiste en construir una aplicación web para una peluquería, donde existen 3 tipos de usuarios: administrador, cliente y trabajador.

Actualizado el 07/10/2026, según el estado actual del proyecto.

1. Administrador

- Crear cuentas de clientes, trabajadores y otros administradores.
- Activar y desactivar cuentas de clientes, trabajadores y administradores. Una cuenta inactiva no puede iniciar sesión, y no se puede desactivar al último administrador activo.
- Ver todos los horarios, o los de un trabajador en particular.
- Editar y eliminar horarios (un horario con citas asociadas no se puede eliminar).
- Eliminar trabajadores (solo si no tienen horarios), clientes (solo si no tienen citas) y otros administradores (siempre debe quedar al menos un administrador activo).
- Iniciar sesión como administrador.

2. Cliente

- Crear su propia cuenta e iniciar sesión.
- Ver los horarios disponibles de los trabajadores.
- Cancelar una cita y reprogramarla a otro horario disponible del mismo trabajador.
- Desactivar su propia cuenta.

3. Trabajador

- Iniciar sesión.
- Ver sus propios horarios.
- Editar sus horarios.
- Eliminar sus horarios.

Requisitos del proyecto

Son 25 requisitos (RF01 a RF25) y los 25 ya tienen su endpoint. El detalle de cada ruta está en el archivo EndPoints.docx.

Requisito	Qué debe hacer	Estado
RF01	Admin crea un cliente	Implementado
RF02	Cliente crea su propia cuenta	Implementado
RF03	Cliente inicia sesión	Implementado
RF04	Admin crea trabajador	Implementado
RF05	Trabajador inicia sesión	Implementado
RF06	Cliente ve horarios	Implementado
RF07	Admin ve horarios	Implementado
RF08	Trabajador ve sus horarios	Implementado
RF09	Trabajador edita horario	Implementado
RF10	Trabajador elimina horario	Implementado
RF11	Admin edita horario	Implementado
RF12	Admin elimina horario	Implementado
RF13	Admin elimina trabajador	Implementado
RF14	Admin elimina cliente	Implementado
RF15.1	Cliente cancela una cita	Implementado
RF15.2	Cliente reprograma una cita	Implementado
RF16	Admin crea admin	Implementado
RF17	Admin inicia sesión	Implementado
RF18	Admin activa cliente	Implementado
RF19	Admin desactiva cliente	Implementado
RF20	Admin activa trabajador	Implementado
RF21	Admin desactiva trabajador	Implementado
RF22	Cliente desactiva su cuenta	Implementado
RF23	Admin desactiva admin	Implementado
RF24	Admin activa admin	Implementado
RF25	Admin elimina admin	Implementado

Entonces, ¿qué aplicación hay que hacer?

Algo parecido a esto:

PELUQUERÍA

ADMIN	CLIENTE	TRABAJADOR
Gestiona usuarios y horarios Crea clientes, trabajadores y admins Activa y desactiva cuentas Edita y elimina horarios Elimina trabajadores, clientes y admins	Consulta horarios disponibles Crea su cuenta Cancela y reprograma citas Desactiva su cuenta	Inicia sesión Ve sus horarios Edita sus horarios Elimina sus horarios

Tecnologías y estructura

Node.js + Express + MySQL + JavaScript

- Librerías: express, mysql2, bcrypt (contraseñas cifradas), cors y dotenv (variables de entorno).
- Arquitectura MVC: carpetas vista (rutas), controlador (validaciones y respuestas) y modelo (consultas SQL).
- Base de datos bdpeluqueria con 4 tablas: clientes, trabajadores (incluye los administradores con rol "admin"), horarios y citas.
- Servidor en el puerto 5020, configurado en el archivo .env.

El proyecto está en un repositorio de GitHub, trabajado en la rama kevin.

Lo más importante

El objetivo principal es aprender a hacer un sistema completo, no solamente una página bonita. Hay que practicar:

Frontend → Backend → Base de datos MySQL

Por ejemplo, el flujo de un cliente:

- Llena el formulario de registro.
- El backend Node.js valida los datos y cifra la contraseña.
- MySQL guarda el cliente.
- Cuando inicia sesión, Node.js busca sus datos en MySQL y compara la contraseña; si es correcta, entra al sistema.

Estado actual y pendientes

- Listo: backend con los endpoints de los 25 requisitos (RF01 a RF25), validaciones y la base de datos de prueba. Incluye el bloqueo de cuentas inactivas en el inicio de sesión y el rechazo (409) de eliminaciones que tienen citas asociadas.
- En avance: frontend inicial en la carpeta frontend (pantalla de inicio de sesión y vistas base de admin, cliente y trabajador). Por ahora solo está conectado el inicio de sesión de administrador.
- Pendiente: endpoints para crear horarios y citas, autenticación con token y control de roles por ruta.

En resumen: a partir de los requisitos funcionales se fue construyendo la base de datos, el backend y las funciones de cada tipo de usuario. Falta conectar el frontend con los tres roles y reforzar la seguridad.